



How LLM Inference Works

— Prompt to Response, Explained —

What actually happens inside a large language model
— in plain human terms.



Chandra Sekhar
AI Engineer & Trainer

What is a Token?

LLMs don't read words — they read **tokens**. A token is roughly a word fragment.

"Inference" → "**Infer**" + "**ence**"

TOKENIZING "HELLO, HOW ARE YOU?"

Hello

,

how

are

you

?

→ **6 tokens**

- ~1 token $\approx$ $\frac{3}{4}$ **of a word** in English
- 100 tokens $\approx$ **75 words**
- GPT-4 has a **128k token** context window



THE PROCESS

Inference = Forward Pass

Your tokens travel through **layers of math** and come out as probabilities for the next token.

YOUR PROMPT

"Explain photosynthesis"



TOKENIZER

[849, 104, 2231...]



N TRANSFORMER LAYERS (x32 TO x96)

Attention + Feed-Forward Math



OUTPUT: PROBABILITY DISTRIBUTION

"Photo" 42% · "Plants" 18%...



Attention: How the Model "Focuses"

For every token, the model asks: "**which other tokens matter most to predict the next word?**"

EXAMPLE: "THE CAT SAT ON THE MAT"

Predicting after "**sat**", the model pays strong attention to "**cat**" — the subject shapes what comes next.

- **Query, Key, Value** — 3 matrices powering attention
- **Multi-head attention** — many "perspectives" in parallel
- Longer context = **more attention pairs** = more compute



How the Model **Picks** Next Words

The model scores every token in its ~50k vocabulary. A **sampling strategy** decides which to output.

Temperature

Low (0.1) = focused.
High (1.5) = creative.

Top-P

Sample from tokens summing to P.
top_p=0.9 cuts unlikely tails.

Top-K

Sample from top K only.
top_k=50 = 50 candidates.

Greedy

Always picks #1 token.
Deterministic, can repeat.



Context Window & KV Cache

Every token gets added back to context. A **KV Cache** saves prior attention computations so the model doesn't redo the work.

Your Prompt

cached ✓

Generated

tokens so far

Next?

predicting...

- Context window = the model's working "**memory**"
- KV Cache = **2-4x speed boost** during generation
- Bigger context = **more RAM/VRAM** required



Prefill vs Decode Phase

Two distinct stages with very different speeds — why your response starts slow then streams fast.

- **PREFILL PHASE**

All input tokens processed **in parallel** in one batch. Fast — the model "reads" your prompt at once.

- **DECODE PHASE**

One token generated **per step**, sequentially. This is the streaming — each word appearing one at a time.



Why Inference Needs GPUs

One forward pass through GPT-4 requires **trillions of operations**. CPUs take minutes. GPUs run in **parallel** — milliseconds.

80GB

VRAM on A100 —
needed for 70B
models

312T

FLOPs/sec — A100
peak throughput

4-bit

Quantization = 4×
smaller

QUANTIZATION = RUN ON YOUR LAPTOP

Compresses weights from 16-bit to 4-bit. Small quality loss, but a **7B model can run on a MacBook**.



When Does Inference Stop?

The model doesn't know when it's "done." It stops based on **stopping criteria**.

- **EOS token** — a special end-of-sentence token it generates
- **max_tokens** — a hard cap you set (e.g. 512)
- **Stop sequences** — custom strings like \n\n
- **User interruption** — you hit stop mid-stream

```
# Calling inference via API
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=512, temperature=0.7,
)
```



YOU NOW UNDERSTAND

LLM Inference

From tokens to attention to KV cache —
you've seen what really happens inside the
box.

Repost

Save

Comment



Follow for more AI breakdowns